

# Thema 2: Moderne Such- und Retrievalverfahren nach Datenstruktur

## TL;DR

**Es gibt keine universelle „beste“ Suchmethode** – die zuverlässigste Architektur unter realen Bedingungen ist **hybride Suche (BM25 + Dense-Embedding, fusioniert per Reciprocal Rank Fusion) mit anschließendem Cross-Encoder-Re-Ranking**; jede Komponente ist einzeln praxiserprobt, unabhängig auf BEIR reproduziert und in Elasticsearch/OpenSearch/Vespa produktiv im Einsatz.

**BM25 bleibt der überraschend robuste Referenzmaßstab** (Kategorie A): Auf dem BEIR-Benchmark schlägt es viele neuronale Dense-Retriever im Zero-Shot-Setting; Dense-Retriever gewinnen erst nach Domänen-Anpassung oder in Kombination mit Re-Ranking. Wer selbst implementiert, sollte BM25 aus Lucene/`rank_bm25` übernehmen und nicht neu erfinden.

**GraphRAG (Microsoft) ist als Kategorie C einzustufen, nicht B**: Es hilft nachweislich bei globalen „Sensemaking“-Fragen, ist aber mit LLM-as-Judge ohne Ground-Truth evaluiert, im Indexaufbau rund 1000× teurer als Vector-RAG und verliert bei präzisen Faktenfragen gegen einfaches RAG. Für die meisten Anwendungen zunächst ignorieren.

## Key Findings

**Klassische strukturierte Suche (Exact/Range/Boolean, B-Tree, SQL) ist unschlagbar**, wo Genauigkeit, Nachvollziehbarkeit und Determinismus zählen – Embeddings sind hier nicht nur überflüssig, sondern schädlich.

**Fuzzy Matching und Record Linkage** sind ein gelöstes, praxisreifes Feld: Levenshtein, n-Gramme, phonetische Verfahren und das probabilistische Fellegi-Sunter-Modell (Splink) skalieren auf Millionen Datensätze.

**Learned Sparse Retrieval (SPLADE)** kombiniert die Vorteile invertierter Indizes mit neuronaler Term-Expansion und generalisiert auf BEIR besser als frühe Dense-Modelle.

**Dense Retrieval** ist durch starke, offen verfügbare Embedding-Modelle (BGE, E5, BGE-M3) produktionsreif; die ANN-Suche darunter (HNSW) ist De-facto-Standard.

**Re-Ranking mit Cross-Encodern** ist der „billigste Präzisionsgewinn“ in RAG-Pipelines, aber mit einem klaren Latenz-/Compute-Trade-off.

**Graph-Retrieval** ist zweigeteilt: klassische Algorithmen (BFS/DFS, Dijkstra, Leiden-Community-Detection) sind ausgereift und reproduzierbar; graphbasierte RAG-Architekturen sind noch unreif.

**Query Routing** zwischen heterogenen Quellen ist die konzeptionell richtige Richtung, aber noch überwiegend Forschungsstand (Kategorie C).

## Details

### 2.1 Structured Search

**Exact Match, Filtering, Range- und Boolean-Queries, Index Lookups**

Die Grundlage strukturierter Suche ist der **B-Tree/B+-Tree-Index** – seit den 1970er-Jahren die Standard-Datenstruktur relationaler Datenbanken (PostgreSQL, MySQL) für Exact-Match- und Range-Queries mit logarithmischer Suchzeit. Für Gleichheitsabfragen sind **Hash-Indizes** noch schneller ( $O(1)$ ), unterstützen aber keine Bereichsabfragen. Diese Verfahren sind Kategorie A: maximal validiert, deterministisch, seit Jahrzehnten produktiv. In Bezug auf Thema 1: PostgreSQL nutzt B-Trees als Default, ClickHouse setzt auf Sparse-Primary-Key-Indizes plus Skip-Indizes für OLAP-Range-Scans.

**Boolean Retrieval** (UND/ODER/NICHT-Verknüpfung von Mengen) ist das älteste IR-Modell und bildet die logische Grundlage sowohl von SQL-`WHERE`-Klauseln als auch invertierter Volltextindizes.

### Query Based Retrieval: Joins, Aggregationen, Query Planning

Komplexe strukturierte Abfragen werden durch den **Query Optimizer** des DBMS realisiert. Kostenbasierte Optimierung (cost-based optimization), Index-Selektion und Join-Reihenfolge-Planung sind ausgereift. Für Analytik referenziert dies auf Thema 1: Spaltenorientierte Systeme (ClickHouse, Snowflake) und Lakehouse-Formate (Iceberg, Delta Lake) optimieren Aggregationen über Predicate-Pushdown und Partition-Pruning.

### Fuzzy Matching, Entity Matching und Record Linkage

**Edit Distance / Levenshtein-Distanz**: Misst die minimale Zahl von Einfüge-/Lösch-/Ersetzungsoperationen zwischen zwei Strings; Grundlage der meisten Tippfehler-Toleranz.

**n-Gramme**: Zerlegung in Zeichenfolgen der Länge  $n$  für approximatives String-Matching; wird von Elasticsearch/OpenSearch und PostgreSQL (`pg_trgm`) genutzt.

**Phonetische Verfahren**: **Soundex** (1918 patentiert), **Metaphone** und **Double Metaphone** kodieren Wörter nach Klang, um phonetisch ähnliche Namen zu finden.

**Record Linkage / Entity Resolution**: Der theoretische Kern ist das **Fellegi-Sunter-Modell** (1969). Die heute führende Open-Source-Implementierung ist **Splink** (UK Ministry of Justice), das das Fellegi-Sunter-Modell mit dem Expectation-Maximization-Algorithmus umsetzt, unüberwacht trainierbar ist und über DuckDB/Spark/Athena-Backends sehr großen Umfang bewältigt. Laut offiziellem README kann Splink „eine Million Datensätze auf einem Laptop in rund einer Minute“ verknüpfen; ein publizierter Benchmark (Robin Linacre) dedupliziert einen Datensatz mit 7 Millionen Datensätzen (über 1 Milliarde paarweise Vergleiche) in gut zwei Minuten und für weniger als 1,00 USD auf AWS EC2. Splink ist Kategorie A (im öffentlichen Sektor, Wissenschaft und Privatwirtschaft breit produktiv eingesetzt, u.a. NHS und britisches Verteidigungsministerium). Alternativen: **dedupe** (Python, aktives Lernen), **Zingg** (Spark), **Python Record Linkage Toolkit**. [Realworlddatascience + 3](#)

↗

### Wann ist klassische strukturierte Suche überlegen?

Strukturierte Suche schlägt semantische/embeddingbasierte Verfahren klar, wenn:

**exakte Übereinstimmung** verlangt wird (IDs, Produktcodes wie `SKU-44819`, Fehlercodes wie `ERR_CONN_RST`) – Dense-Retriever liefern hier semantisch benachbarten, aber falschen Müll; [Big Data Boutique](#)

↗

**Determinismus und Nachvollziehbarkeit** juristisch/regulatorisch gefordert sind; **numerische Bereiche, Filter, Aggregationen** im Vordergrund stehen;

**kein Trainingsdaten-/Compute-Budget** für Embedding-Modelle existiert;  
**strukturierte Metadaten** (Datum, Kategorie, Preis) gefiltert werden – hier ist Metadata-Filtering vor/mit Vektorsuche der Standard.

## 2.2 Unstructured Search und Information Retrieval

### Keyword / Lexical Search: Inverted Index, TF-IDF, BM25

Der **invertierte Index** (Term → Liste der Dokumente) ist die Grundstruktur aller Volltextsuche (Lucene, Elasticsearch, OpenSearch, Solr). **TF-IDF** gewichtet Terme nach Häufigkeit im Dokument und Seltenheit im Korpus.

**BM25** (Best Matching 25) ist der Goldstandard des lexikalischen Rankings. Die maßgebliche Referenz ist Robertson & Zaragoza, *The Probabilistic Relevance Framework: BM25 and Beyond*, Foundations and Trends in Information Retrieval, Vol. 3(4), 2009, DOI 10.1561/15000000019. BM25 erweitert TF-IDF um Term-Sättigung (Parameter  $k_1$ ) und Dokumentlängen-Normalisierung (Parameter  $b$ ). Die Ursprünge liegen im Okapi-System (Robertson et al., TREC-1/TREC-3, frühe 1990er). **BM25F** erweitert das Modell auf strukturierte Dokumente mit gewichteten Feldern. [ResearchGate](#)

↗  
[Now Publishers](#)

↗  
**Evidenzbewertung BM25 (Kategorie A):** Auf **BEIR** (Thakur et al., NeurIPS 2021 Datasets & Benchmarks, arXiv:2104.08663) erweist sich BM25 als „robuster Baseline“, der viele neuronale Modelle im Zero-Shot-Setting über die 18 Datensätze hinweg übertrifft. Bestätigt durch die unabhängige Reproduktionsstudie Kamaloo et al., *Resources for Brewing BEIR*, SIGIR 2024 (arXiv:2306.07471, DOI 10.1145/3626772.3657862). Implementierung: Lucene/Anserini (produktiv), `rank_bm25` (Python, für Prototyping), `bm25s` (schnelle Python-Implementierung). [arxiv + 2](#)

↗

### Sparse Retrieval: Learned Sparse Retrieval / SPLADE

**SPLADE** (Sparse Lexical and Expansion Model) von Formal, Piwowarski & Clinchant, SIGIR 2021 (v1, DOI 10.1145/3404835.3463098) und SPLADE v2/v3 (Naver Labs Europe) lernt sparse Term-Gewichte über den BERT-Masked-Language-Model-Kopf mit Sparsity-Regularisierung. Vorteile gegenüber Dense: nutzt den effizienten invertierten Index, bietet explizites lexikalisches Matching und Interpretierbarkeit, und generalisiert nachweislich besser auf Out-of-Domain-Daten (BEIR). [GitHub](#)

↗

**Evidenzbewertung SPLADE (Kategorie B):** Auf mehreren Benchmarks bestätigt (MS MARCO in-domain, BEIR out-of-domain), offizielle Implementierung ([github.com/naver/splade](#)), in Suchmaschinen wie Vespa/Elasticsearch integrierbar. Aktuelle Baseline: **SPLADE-v3** (Lassance et al., 2024, arXiv:2403.06789). Praktische Einschränkung: Query-Latenz kann durch Term-Expansion höher als bei reinem BM25 ausfallen; benötigt GPU für Encoding. [arxiv](#)

↗

### Dense Retrieval: Bi-Encoder, Embeddings, ANN-Suche

Das Paradigma wurde durch **DPR (Dense Passage Retrieval)**, Karpukhin et al., EMNLP 2020 (arXiv:2004.04906, DOI 10.18653/v1/2020.emnlp-main.550) etabliert: Ein Dual-/Bi-Encoder (getrennte

BERT-Encoder für Query und Passage) bildet beide in einen gemeinsamen Vektorraum ab; Ähnlichkeit ist das Skalarprodukt. Laut Abstract übertrifft der Dense-Retriever „ein starkes Lucene-BM25-System deutlich um 9–19 % absolute Top-20-Passage-Retrieval-Genauigkeit“. **Kritischer Hinweis:** Eine spätere Replikationsstudie (Ma et al., arXiv:2104.05740) zeigt mit einer besser abgestimmten Pyserini-BM25-Baseline rund 7 Punkte höhere BM25-Werte als im Original – der reale Abstand ist also kleiner als ursprünglich berichtet; ein typisches Beispiel dafür, dass die Baseline in Originalpapers oft zu schwach gewählt ist. Der konzeptuelle Vorläufer für Satz-Embeddings ist **Sentence-BERT** (Reimers & Gurevych, EMNLP 2019). [arxiv](#)

## Aktuelle Embedding-Modelle (Kategorie B, teils A):

**BGE** (BAAI General Embedding) und **E5** (Microsoft) – offene, starke Encoder-Modelle.

**BGE-M3** (Chen et al., 2024, arXiv:2402.03216) – vereint dense, sparse und multi-vector Retrieval in einem Modell, unterstützt 100+ Sprachen und bis zu 8192 Token. Empfohlene Pipeline laut Autoren: Hybrid-Retrieval + Re-Ranking. [arXiv + 2](#)

**E5-mistral-7b** (Wang et al., 2024) – LLM-basiertes Embedding, führte zeitweise MTEB an.

Kommerziell: **OpenAI text-embedding-3-large**, **Cohere Embed v3**.

Der neutrale Vergleichsmaßstab ist **MTEB** (Massive Text Embedding Benchmark), Muennighoff et al., 2022/2023 (arXiv:2210.07316), mit 56+ Datensätzen über 8 Aufgabentypen und öffentlichem Leaderboard.

**Kritischer Hinweis zur Evidenz:** Der MTEB-Headline-Score ist ein ungewichteter Durchschnitt und wird häufig fehlinterpretiert; das Retrieval-Subset (bzw. BEIR-Werte) ist für Suchanwendungen aussagekräftiger als der Gesamtscore. Ein Modell, das auf MTEB-Trainingsdaten feinabgestimmt wurde, kann auf dem Leaderboard glänzen, ohne besser zu generalisieren. [arxiv](#)

[Tensoria](#)

**ANN-Suche (Approximate Nearest Neighbor):** Die darunterliegende Indexstruktur ist fast immer **HNSW** (Hierarchical Navigable Small World), Malkov & Yashunin, arXiv:1603.09320, publiziert in IEEE TPAMI 42(4):824–836, DOI 10.1109/TPAMI.2018.2889473 (online first Dez. 2018, Heft April 2020 – daher variierende Jahresangaben 2016/2018/2020). HNSW baut inkrementell eine mehrschichtige, navigierbare Small-World-Graphstruktur auf, in der die Suche in der obersten (grobsten) Schicht startet und sich nach unten verfeinert, was näherungsweise logarithmische Suchkomplexität ermöglicht. Es ist die Grundlage nahezu aller Vektordatenbanken (FAISS, Milvus, Qdrant, Weaviate, pgvector – siehe Thema 1). Schlüsselparameter: M (Verbindungen pro Knoten) und ef\_search (Suchbreite). [arxiv](#)

## Hybrid Search: Score Fusion und Reciprocal Rank Fusion

Reine Vektorsuche hat einen blinden Fleck bei exakten Tokens (Codes, Namen); BM25 versagt bei Paraphrasen und Synonymen. **Hybride Suche** kombiniert beide. Das Problem: BM25-Scores und Kosinus-Ähnlichkeiten liegen auf unvergleichbaren Skalen. [Big Data Boutique](#)

**Reciprocal Rank Fusion (RRF)** von Cormack, Clarke & Büttcher, SIGIR 2009 (DOI 10.1145/1571941.1572114) löst dies elegant: Es ignoriert die Roh-Scores und nutzt nur die Rangpositionen – jedes Dokument erhält  $\text{Score} = \sum 1/(k + \text{rang})$ , mit k typischerweise 60. Kein Training, keine Score-Normalisierung, ca. sieben Zeilen

Code. [LangChain4j](#)

↗

[LanceDB](#)

↗

**Evidenzbewertung RRF (Kategorie A):** Im Originalpaper schlug RRF Condorcet Fuse, CombMNZ und Learning-to-Rank-Methoden auf LETOR 3. Heute der Standard-Fusionsmechanismus in **Elasticsearch** (rrf-Retriever, Parameter `rank_constant`), **OpenSearch**, **Weaviate**, **Qdrant** und **Azure AI Search**.

Praktischer Vorteil hybrider Suche mit Benchmark-Evidenz: robuster über heterogene Query-Typen als jede Einzelmethode. Alternative: gewichtete Linearkombination normalisierter Scores (erfordert aber Kalibrierung/Tuning). [Serghei's Blog](#)

↗

## Re-Ranking: Cross-Encoder, Multi-Stage Retrieval

Nach der Kandidatengenerierung (Recall-orientiert) folgt eine Präzisionsstufe. Ein **Cross-Encoder** verarbeitet Query und Dokument gemeinsam in einem einzigen Transformer-Forward-Pass und erfasst dadurch feinere Relevanz als ein Bi-Encoder – ist aber zu teuer für die Erststufe (ein Pass pro Kandidat).

[Big Data Boutique](#)

↗

[arXiv](#)

↗

## Wichtigste Re-Ranker (Kategorie B, teils A):

**MiniLM-Cross-Encoder** (ms-marco-MiniLM-L-6-v2) – in BEIR als bestes öffentliches Cross-Encoder-Modell identifiziert; leichtgewichtig, aus `sentence-transformers` verfügbar. [Neurips](#)

↗

**BGE-reranker-v2-m3** (BAAI) – offener, multilingualer Cross-Encoder zum Self-Hosting.

**Cohere Rerank** – gehostete API, geringster Integrationsaufwand.

**ColBERT / ColBERTv2** (Khattab & Zaharia, SIGIR 2020, arXiv:2004.12832; Santhanam et al. 2021, arXiv:2112.01488) – **Late Interaction**: Token-Level-Embeddings mit MaxSim-Operation; Mittelweg zwischen Bi- und Cross-Encoder, deutlich schneller als Cross-Encoder bei nahezu vergleichbarer Qualität. In BEIR übertrifft ColBERT die Single-Vector-Systeme DPR/ANCE. [arxiv](#)

↗

**Trade-off Qualität vs. Latenz/Compute:** BEIR zeigt, dass Re-Ranking- und Late-Interaction-Modelle im Schnitt die besten Zero-Shot-Ergebnisse liefern – „however, at high computational costs“. Praxisrichtwerte für Re-Ranking von ~100 Kandidaten: MiniLM-Klasse ca. 50–80 ms, BGE-v2-m3 (568M) ca. 80–200 ms auf GPU (A10/L4-Klasse). Ein typischer Uplift liegt bei rund 0,05–0,08 nDCG@10, wobei Re-Ranking bei bereits hohem Recall@3 kaum noch hilft und bei Domänen-Shift sogar schaden kann. **Kritische Einordnung:** Re-Ranker nur mit einem eigenen Golden-Set (50–200 gelabelte Query-Dokument-Paare) einführen und nDCG@10 vorher/nachher messen – nicht blind aus Tutorials übernehmen. [arxiv + 3](#)

↗

## Retrieval Pipelines: mehrstufige Architekturen

Das dominante produktive Muster ist: **Query → (parallel) BM25 + Dense-Retrieval → RRF-Fusion → Cross-Encoder-Re-Ranking → Top-k → LLM/Ergebnis**, optional mit Metadata-Filtering vorgeschaltet.

**Elasticsearch/OpenSearch:** invertierter Index + k-NN-Plugin (HNSW) + rrf-Retriever + Reranker; verbreitetster Stack.

**Vespa**: vereint lexikalische, Vektor- und strukturierte Suche plus mehrphasiges Ranking in einer Engine; nutzt WAND/nearestNeighbor-Operatoren. Produktiv u.a. bei **Perplexity** (Retrieval über Milliarden Dokumente) und **Vinted**: Laut Vinted-Engineering-Blog (Sept. 2024, „Goodbye Elasticsearch, Hello Vespa“) umfasst der Index rund 1 Milliarde aktiv durchsuchbare Objekte; die Migration halbierte die Serverzahl (auf 60), verbesserte die Such-Latenz um das 2,5-Fache und die Indexierungs-Latenz um das 3-Fache und senkte die Sichtbarkeitszeit von Änderungen von 300 s auf 5 s (Spitzenlast ~20.000 req/s unter 150 ms p99). [Vespa + 2](#)

↗

## 2.3 Graph Search und Graph Retrieval

### Graph Traversal

**BFS** und **DFS** sind die fundamentalen Traversierungsverfahren; **Dijkstra** und **A\*** für Shortest-Path, **Bellman-Ford** bei negativen Kanten. Multi-Hop-Traversal, Path-Queries und Reachability sind die Kernoperationen von Graph-Datenbanken (Neo4j mit Cypher – siehe Thema 1). Kategorie A: klassische, bewiesene Algorithmen, verfügbar in NetworkX, igraph, Neo4j GDS.

### Subgraph Matching

**Subgraph-Isomorphie** ist NP-vollständig, praktisch aber durch Algorithmen wie VF2 und Ullmann für moderate Graphgrößen lösbar. **Graph Pattern Matching** ist deklarativ über Cypher/Gremlin/SPARQL zugänglich. **Motif Search** (Finden häufiger kleiner Teilgraphen) ist in Bioinformatik und Netzwerkanalyse etabliert.

### Community Detection: Louvain und Leiden

**Louvain** (Blondel et al., 2008) maximiert Modularität greedy und ist weit verbreitet, hat aber einen Defekt: Es kann schlecht verbundene oder sogar unzusammenhängende Communities erzeugen. **Leiden** (Traag, Waltman & van Eck, *From Louvain to Leiden: guaranteeing well-connected communities*, Scientific Reports 9:5233, 2019, DOI 10.1038/s41598-019-41695-z) behebt dies durch eine Verfeinerungsphase und garantiert zusammenhängende Communities; im Originalpaper beobachten die Autoren, dass „bis zu 25 % der Communities schlecht verbunden und bis zu 16 % unzusammenhängend“ sind (z. B. DBLP-Netzwerk 16 % disconnected; Amazon-Netzwerk 5 % disconnected, aber 25 % badly connected). Leiden ist zudem schneller als Louvain. [arxiv + 3](#)

↗

**Evidenzbewertung Leiden (Kategorie A)**: Mehrfach unabhängig reproduziert, Standard in igraph, NetworkX, cuGraph (GPU), breit in Genomik/Einzelzell-Analyse und in GraphRAG (für Community-Summaries) eingesetzt. [PubMed Central](#)

↗

### Graph Embeddings

**Node Embeddings: DeepWalk** (Perozzi et al., KDD 2014) und **Node2Vec** (Grover & Leskovec, KDD 2016) übertragen das word2vec-Skip-Gram-Modell auf Graphen via (biased) Random Walks. Node2Vec balanciert über zwei Parameter (p, q) zwischen BFS-artiger (Homophilie) und DFS-artiger (strukturelle Äquivalenz) Exploration. Kategorie A/B: skalierbar, breit reproduziert, für Link-Prediction und Node-

Classification bewährt. [arxiv](#)

↗

[arxiv](#)

↗

**Knowledge Graph Embeddings: TransE** (Bordes et al., 2013) modelliert Relationen als Translationen im Vektorraum; Nachfolger TransH, TransR, DistMult, ComplEx, RotatE. Für Link-Prediction in Wissensgraphen.

**Graph Neural Networks: GraphSAGE** (Hamilton et al., 2017) ist der praktisch relevanteste induktive Ansatz (generalisiert auf ungesehene Knoten); GCN und GAT sind weitere Standardarchitekturen. Praktisch relevant, aber höherer Implementierungs-/Trainingsaufwand (PyTorch Geometric, DGL).

## **Semantic Web: RDF, SPARQL, Triple Stores, Ontologien**

**RDF** modelliert Wissen als Tripel (Subjekt, Prädikat, Objekt), abgefragt mit **SPARQL**. **Triple Stores** (produktiv: **Virtuoso**, **GraphDB**, **RDFox**, **Amazon Neptune**, **Stardog**, **Apache Jena**) unterstützen expressive Queries und **Inferenz** über Ontologien (RDFS/OWL). Vorteile: Schema-on-read, reiche Semantik, Wissensinferenz, einfache Updates. Unabhängige Benchmarks (ESWC 2023, Wikidata; DBpedia SPARQL Benchmark) zeigen: RDFox und GraphDB sind die schnellsten (gefolgt von Amazon Neptune), aber alle Triple Stores kämpfen mit Skalierung über den RAM hinaus und mit komplexen Queries (FILTER, OPTIONAL, DISTINCT), die Timeouts verursachen. [uspto + 4](#)

↗

## **Wann welches Graph-Verfahren?**

**Klassische Graphalgorithmen** (BFS/DFS/Shortest Path/Leiden): wenn die Frage explizit strukturell ist (Pfade, Erreichbarkeit, Cluster) und der Graph vorliegt.

**Deklarative Graph-Queries** (Cypher/SPARQL): wenn komplexe Muster über bekannte Schemata/Ontologien abgefragt werden und Nachvollziehbarkeit/Inferenz zählen.

**Embeddingbasierte Verfahren** (Node2Vec/GNN): wenn Ähnlichkeit, Link-Prediction oder Node-Classification gefragt sind und Trainingsdaten existieren – besonders zur Kombination von Graphstruktur mit semantischer Ähnlichkeit.

## **2.4 Zusammenspiel und moderne Retrievalarchitekturen**

### **Kombinationsmuster**

**Keyword + Vector (Hybrid)**: siehe 2.2, der Produktions-Default.

**Metadata Filtering + Vector Search**: Vorfilterung nach strukturierten Attributen (Datum, Kategorie, Berechtigung) plus semantische Suche – der häufigste Enterprise-Fall; alle Vektor-DBs (Milvus, Qdrant, pgvector) unterstützen dies nativ.

**Graph + Vector Search**: Kombination von Graphtraversierung und Embedding-Ähnlichkeit; konzeptionell attraktiv, praktisch noch jung.

**Structured + Semantic**: Query Routing entscheidet, welcher Teil einer Query an SQL und welcher an semantische Suche geht.

### **Retrieval Augmented Generation (RAG)**

**RAG** wurde von Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, NeurIPS 2020 (arXiv:2005.11401) eingeführt: Ein neuronaler Retriever (ursprünglich DPR) liefert Passagen, die ein seq2seq-Generator (BART) als nicht-parametrisches Gedächtnis nutzt. Das Standard-Architekturmuster heute: **Chunking → Embedding/Index → (Hybrid-)Retrieval → Re-Ranking → Kontextinjektion → LLM-Generierung**. RAG ist Kategorie A als Architekturparadigma (allgegenwärtig produktiv), wobei die Qualität vollständig von der Retrieval-Qualität abhängt – deshalb ist dieses Thema für RAG zentral. [arXiv](#)

↗

## **GraphRAG – kritische Evidenzbewertung (Kategorie C, nicht B)**

**GraphRAG** (Edge, Trinh, Cheng, Bradley, Chao, Mody, Truitt, Metropolitansky, Ness & Larson, *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*, Microsoft Research, 2024, arXiv:2404.16130) baut einen LLM-extrahierten Entity-Knowledge-Graph auf, clustert ihn per Leiden und generiert vorab Community-Summaries. Ziel sind „global sensemaking“-Fragen („Was sind die Hauptthemen?“), die klassisches RAG schlecht beantwortet. [Microsoft](#)

↗

[Microsoft](#)

↗

## **Warum nur Kategorie C:**

**Schwache Evaluationsmethodik:** Die Original-Evaluation nutzte **LLM-as-Judge (GPT-4) ohne Ground-Truth** auf LLM-generierten Fragen mit den Metriken Comprehensiveness, Diversity und Empowerment – **keine klassischen IR-Metriken (nDCG, Recall) und keine standardisierten QA-Benchmarks mit Exact-Match/F1**. Reportete Win-Rates lagen bei 72–83 % (Comprehensiveness) bzw. 62–82 % (Diversity) gegen naives RAG. LLM-as-Judge-Summarization-Evaluation leidet nachweislich unter Positions-/Reihenfolge-Bias. [Microsoft + 2](#)

↗

**Unfairer Baseline-Vergleich:** Der Vergleich naive RAG (kleines Chunk-Fenster) vs. GraphRAG global (Zugriff auf aggregierten Gesamtkorpus) ist strukturell asymmetrisch; NodeRAG (Xu et al., arXiv:2504.11544) kritisiert die nicht standardisierten Prompts und Retrieval-Token-Mengen und misst für GraphRAG Global Search Query-Latenzen von über 20 Sekunden.

**Hohe Kosten:** Laut Microsofts eigener LazyGraphRAG-Ankündigung (Research-Blog, Nov. 2024) sind „die Datenindexierungskosten von LazyGraphRAG identisch mit Vector RAG und betragen 0,1 % der Kosten von vollem GraphRAG“ – vollständiges GraphRAG ist im Indexaufbau also rund **1000× teurer** als Vector-RAG. Bei Global-Queries erreicht LazyGraphRAG vergleichbare Qualität „bei mehr als 700-fach niedrigeren Query-Kosten“; für 4 % der Query-Kosten von GraphRAG Global Search übertrifft es alle Vergleichsmethoden.

**Unabhängige Reproduktion widerspricht dem Hype:** Han et al., *RAG vs. GraphRAG: A Systematic Evaluation and Key Insights*, ACM SIGKDD 2026 (arXiv:2502.11371, DOI 10.1145/3770855.3817575) finden **keinen konsistenten Gewinner**: RAG gewinnt bei Single-Hop- und detail-orientierten Faktenfragen (unter Ground-Truth-Metriken), GraphRAG bei Multi-Hop-/Reasoning- und korpus-weiten Summaries. Bei Ground-Truth-Evaluation (statt LLM-as-Judge) unterperformt community-basiertes GraphRAG/Global-Search generell gegenüber Standard-RAG; nur etwa 65,8 % der Antwort-Entitäten waren überhaupt im konstruierten Knowledge-Graph enthalten. [arXiv](#)

↗

**Fazit GraphRAG:** Nützliches Werkzeug für einen Nischenfall (globale Sensemaking-/Summarization-Fragen über private Korpora), aber teuer, unreif evaluiert und für den Standard-Faktenabruf unterlegen.

Für die meisten Anwendungen zunächst ignorieren; wenn Graph-Struktur gebraucht wird, kostengünstigere Varianten (LazyGraphRAG, LightRAG, NodeRAG) prüfen.

### Query Routing und Federated Search (Kategorie C)

Die zentrale Architekturfrage: **universelle Suchmethode vs. Query-abhängiges Routing?** Der Forschungstrend geht klar zu Routing. Ansätze: LLM-as-Router (dispatcht Sub-Queries an Quellen), leichtgewichtige neuronale Klassifikatoren (**RAGRoute**, arXiv:2502.19280 – bis zu 80,65 % weniger Kommunikationsvolumen und 52,50 % weniger Latenz bei gehaltener Genauigkeit), regelbasierte Router. **Kritische Einordnung:** Diese Systeme sind überwiegend Forschungsstand (2024–2026), oft nur autorenevaluiert. Der pragmatische Produktions-Kompromiss ist häufig **parallele Retrieval über mehrere Quellen + Fusion** statt fehleranfälligem hartem Routing, wenn Quellgrenzen mehrdeutig sind (RealRoute-Argument, arXiv:2604.20860). Einfaches, regelbasiertes Routing (Fehlercodes/IDs → BM25/SQL; natürlichsprachliche Fragen → Dense) ist dagegen praxistauglich und billig. [arxiv + 2](#)

## Empirischer Vergleich der Kernverfahren

| Verfahren Kat.        |     | Qualität<br>(BEIR/nDCG)              | Latenz Compute/Daten |                       | Skalierung         | Impl.-Aufwand             | Evidenz                           |
|-----------------------|-----|--------------------------------------|----------------------|-----------------------|--------------------|---------------------------|-----------------------------------|
| BM25                  | A   | Robuster Baseline, oft > frühe Dense | Sehr niedrig         | Minimal, kein GPU     | Exzellent (Lucene) | Aus Bibliothek            | Unabhängig reprod (BEIR, Kamalle) |
| SPLADE v3             | B   | > BM25 auf BEIR-Schnitt              | Mittel               | GPU für Encoding      | Gut (inv. Index)   | Modell + Integration      | Mehrere Benchr                    |
| Dense (BGE/E5)        | B   | Stark nach Domänen-Fit               | Niedrig (ANN)        | GPU, ggf. Fine-Tuning | Sehr gut (HNSW)    | sentence-transformers     | MTEB/produkt                      |
| BGE-M3                | B   | Stark, multilingual                  | Niedrig-mittel       | GPU                   | Sehr gut           | FlagEmbedding             | Autore breite Adopti              |
| Cross-Encoder Re-Rank | A/B | Höchste Präzision                    | Hoch                 | GPU (pro Kandidat)    | Nur Top-k          | sentence-transformers/API | BEIR („high co                    |
| ColBERTv2             | B   | Nahe Cross-Encoder                   | Mittel               | GPU, großer Index     | Gut (PLAID)        | Offizielles Repo          | BEIR/L reprod                     |
| Hybrid + RRF          | A   | Robuster als Einzelmethode           | Niedrig-mittel       | Kombiniert            | Sehr gut           | Elastic/OpenSearch nativ  | Produkt Default                   |

| Verfahren Kat.    | Qualität<br>(BEIR/nDCG)     | Latenz       | Compute/Daten           | Skalierung    | Impl.-Aufwand | Evid              |
|-------------------|-----------------------------|--------------|-------------------------|---------------|---------------|-------------------|
| <b>GraphRAG C</b> | Nur<br>global/summarization | Sehr<br>hoch | ~1000× Index-<br>Kosten | Problematisch | Komplex       | LLM-Ju<br>umstrit |

## Storyline für eine Präsentation

**Problem:** Nutzer stellen heterogene Anfragen (exakte IDs, Stichwörter, natürlichsprachliche Fragen) über heterogene Daten (Tabellen, Text, Graphen). Keine Einzelmethode deckt alles ab.

**Bisherige Lösung:** Boolean/TF-IDF/BM25 über invertierte Indizes – robust, aber blind für Semantik/Synonyme.

**Aktuelle funktionierende Verfahren:** Dense Retrieval (BGE/E5) über HNSW; Learned Sparse (SPLADE); Late Interaction (ColBERT); Cross-Encoder-Re-Ranking.

**Technische Unterschiede:** sparse vs. dense Repräsentation; Bi- vs. Cross- vs. Late-Interaction; Rang- vs. Score-Fusion.

**Empirischer Vergleich:** BEIR (neutral, zero-shot) zeigt BM25 als starken Baseline, Re-Ranking/Late-Interaction als Qualitätsspitze bei hohen Kosten; Kamaloo-Reproduktion bestätigt.

**Praktische Bewertung:** Hybrid+RRF+Rerank als robuster Default; GraphRAG kritisch einordnen.

**Empfehlung:** gestaffelter Aufbau, immer gegen eigenes Golden-Set gemessen.

## Recommendations

### Die 3–5 Verfahren, die man wirklich kennen muss

**BM25** (+ invertierter Index) – der unverzichtbare Baseline. Jede Evaluation beginnt hier.

**Hybride Suche mit RRF** (BM25 + Dense-Embedding) – der zuverlässigste Produktions-Default.

**Dense Retrieval mit modernen Embeddings** (BGE/E5/BGE-M3) über HNSW.

**Cross-Encoder-Re-Ranking** (MiniLM/BGE-reranker/Cohere) – der günstigste Präzisionsgewinn.

**SPLADE** (learned sparse) und **ColBERT** (late interaction) als fortgeschrittene Optionen, wenn Hybrid+Rerank nicht reicht.

**Was gilt als besonders relevant / funktioniert am zuverlässigsten:** Die Kombination **Hybrid + RRF +**

**Cross-Encoder-Re-Ranking** – jede Komponente ist einzeln unabhängig validiert und produktiv im Einsatz.

**Am besten zum Selbst-Implementieren** eignet sich BM25 (bzw. das Zusammenschalten der Pipeline aus Bibliotheken); **für den praktischen Einsatz** ist der Elasticsearch/OpenSearch/Vespa-Stack am ausgereiftesten.

### Konkrete gestaffelte Empfehlung

**Stufe 1 (Start, jede Anwendung):** BM25 aus Lucene/OpenSearch aufsetzen, mit einem Golden-Set (50–200 gelabelte Query-Doc-Paare) nDCG@10 messen. Das ist die Baseline, die jedes komplexere Verfahren schlagen muss. **Stufe 2 (semantischer Bedarf):** Hybrid-Suche ergänzen – Dense-Embedding (BGE-large

oder BGE-M3, bei kommerziell/Support-Bedarf OpenAI text-embedding-3) + BM25, fusioniert per RRF. In Elasticsearch/OpenSearch/Vespa nativ verfügbar. **Stufe 3 (Präzisionsbedarf):** Cross-Encoder-Re-Ranking der Top-20–50 einführen – zuerst gehostet (Cohere) zum schnellen Test, dann self-hosted (bge-reranker-v2-m3) für Kostenkontrolle. Nur behalten, wenn das Golden-Set einen messbaren Uplift zeigt. **Stufe 4 (Skalierung/Spezialfälle):** SPLADE oder ColBERTv2 evaluieren; Query-Routing nur bei klar getrennten heterogenen Quellen und nur regelbasiert beginnen. **Benchmarks/Schwellen, die die Empfehlung ändern:** Steigt Recall@3 bereits über ~0,9, lohnt Re-Ranking kaum noch. Zeigt das Golden-Set bei Domänen-Shift einen nDCG-Rückgang durch den Re-Ranker, diesen entfernen oder auf 1k–5k Domänenpaaren fine-tunen. Bei genuin globalen Summarization-Fragen über ein privates Korpus GraphRAG-Varianten (LazyGraphRAG) testen. [CustomGPT](#)

## Was man selbst implementieren vs. übernehmen sollte

**Übernehmen (nie selbst bauen):** BM25 (Lucene/[\[rank\\_bm25\]](#)/[\[bm25s\]](#)), HNSW (FAISS/hnswlib/Vektor-DB), Embedding-Modelle (sentence-transformers/FlagEmbedding), Cross-Encoder, RRF (in Suchmaschine integriert), Record Linkage (Splink).

**Selbst implementieren (lohnt sich):** die **Fusions- und Routing-Logik** der Pipeline, das **Evaluations-Harness mit Golden-Set**, und domänenspezifisches **Fine-Tuning** von Embedding/Reranker (1k–5k gelabelte Paare) – dort liegt der eigentliche anwendungsspezifische Mehrwert. [Big Data Boutique](#)

## Was man (zunächst) ignorieren kann

**GraphRAG** in seiner vollen Form – zu teuer, unreif evaluiert, nur Nischennutzen; falls Graph-Bedarf besteht, LazyGraphRAG/LightRAG/NodeRAG prüfen.

**Knowledge-Graph-Embeddings/GNNs** für reine Textsuche – nur relevant bei genuinen Graphproblemen.

**Komplexe agentische Query-Router** – Forschungsstand; mit einfachem regelbasiertem Routing oder paralleler Fusion beginnen.

**Reine Vektorsuche ohne lexikalischen Anteil** als alleinige Lösung – der dokumentierte blinde Fleck bei exakten Tokens macht sie für viele reale Queries fragil.

## Übergreifende Synthese: Thema 1 (Datenbanken) ↔ Thema 2 (Retrieval)

Datenmodell und Datenbankarchitektur determinieren das sinnvolle Retrievalverfahren weitgehend. Eine praktische Entscheidungslogik:

**Datenart → Datenmodell → Datenbanksystem → Query-Typ → Retrievalverfahren → Ranking → Ergebnis**

**Strukturierte, tabellarische Daten** (Transaktionen, Kennzahlen) → relationales/spaltenorientiertes Modell → PostgreSQL/ClickHouse/Snowflake → exakte/Range-/Aggregations-Queries → **B-Tree-Index-Lookup / SQL-Query-Optimizer** → deterministische Sortierung → exaktes Ergebnis. *Kein Embedding nötig.*

**Unstrukturierter Text** (Dokumente, Support-Tickets) → invertierter Index + Vektorindex → Elasticsearch/OpenSearch/Vespa + Milvus/Qdrant/pgvector → Volltext-/semantische Queries → **Hybrid**

**(BM25 + Dense) + RRF → Cross-Encoder-Re-Ranking** → relevanzsortierte Passagen (ggf. in RAG).  
**Verknüpfte/relationale Entitäten** (Wissensgraphen, Netzwerke) → Graph-/RDF-Modell → Neo4j/Virtuoso/GraphDB → Pfad-/Muster-/Reachability-Queries →  
**Graphtraversierung/Cypher/SPARQL**, ggf. + Node-Embeddings → strukturbasiert → Teilgraphen/Pfade.  
**Gemischte Anfragen** → mehrere Systeme → **Metadata-Filtering + Vektorsuche** oder (regelbasiertes)  
**Query Routing** → quellenspezifisches Retrieval + Fusion → einheitliches Ergebnis.

**Kernaussage:** Die in Thema 1 getroffene Wahl des Datenmodells legt den Query-Typ fest, und der Query-Typ legt das Retrievalverfahren fest. HNSW als Standard-Vektorindex (Thema 1) ist genau die Brücke: Es ist die physische Datenstruktur, die Dense Retrieval (Thema 2) überhaupt skalierbar macht. Die praktisch robusteste Gesamtarchitektur ist selten „eine Methode für alles“, sondern eine **mehrstufige Pipeline, die strukturierte Filterung, lexikalische und semantische Suche kombiniert und mit einem Re-Ranker abschließt** – wobei man mit der einfachsten Stufe beginnt und Komplexität nur gegen gemessenen Nutzen hinzufügt. [Big Data Boutique](#)

↗

## Caveats

**Benchmark-Skepsis:** Wie bei den Datenbank-Herstellerbenchmarks aus Thema 1 sind auch hier viele SOTA-Behauptungen optimistisch. Ein dokumentiertes Beispiel: Der berühmte „9–19 %“-Vorsprung von DPR über BM25 schrumpft in der Replikationsstudie um ~7 Punkte, sobald die BM25-Baseline sauber abgestimmt ist. BEIR (Zero-Shot) und die Kamaloo-Reproduktionsstudie sind die neutralsten Maßstäbe; MTEB-Headline-Scores dagegen leicht durch Trainingsdaten-Leakage verzerrbar. Immer auf einem eigenen, domänenspezifischen Golden-Set nachmessen.

**LLM-as-Judge** (v.a. bei GraphRAG und RAG-Evaluationen) ist anfällig für Positions-/Reihenfolge-Bias und liefert ohne Ground-Truth keine verlässlichen Relevanzurteile.

Einige der zitierten Vergleichs- und Routing-Arbeiten (RealRoute, NodeRAG, diverse „RouteRAG“-Varianten) tragen arXiv-Datumsstempel aus 2025/2026 und sind teils nur autorenevaluiert bzw. noch nicht peer-reviewed – als Trendindikatoren, nicht als gesicherte Evidenz zu lesen.

Die konkreten Latenzzahlen für Re-Ranker sind hardwareabhängige Richtwerte (GPU-Klasse A10/L4) und variieren mit Kandidatenzahl und Passagenlänge.

Kostenzahlen zu GraphRAG stammen aus Microsofts eigenem Research-Blog (0,1 %/700×, direkt belegbar); kursierende absolute Dollar-Beträge aus Sekundärquellen sind mit Vorsicht zu behandeln.